TECHNICAL REQUIREMENTS DOCUMENT (TRD)

Mall & Online Offers Aggregator Mobile Application

---

1. Document Overview

1.1 Purpose

This document defines the **technical architecture, system components, integration details, database design, security mechanisms, and deployment plan** for the Mall & Online Offers Aggregator Mobile Application.

---

2. System Architecture

2.1 Architecture Style

The system follows a:

- Client-Server Architecture
- RESTful API-based Communication
- Microservice-ready backend structure

2.2 High-Level Architecture

Mobile App (React Native)
⬇
REST API (Spring Boot Backend)
⬇
MongoDB Database

Additional Services:

- Socket.io for real-time notifications
- Payment Gateway (Razorpay / Stripe)
- Firebase Cloud Messaging (Push Notifications)

---

3. Technology Stack

3.1 Frontend

- Framework: React Native (Expo)

- Language: JavaScript / TypeScript
- Navigation: React Navigation
- State Management: Context API / Redux (optional)
- HTTP Client: Axios
- Real-time: socket.io-client

3.2 Backend

- Framework: Spring Boot
- Language: Java
- API Style: REST
- Security: Spring Security + JWT
- Real-time: Socket.io Server
- Build Tool: Maven / Gradle

3.3 Database

- Database Type: NoSQL
- Database: MongoDB
- ORM: Spring Data MongoDB

3.4 Hosting & Deployment

- Backend Hosting: AWS EC2 / Render
- Database Hosting: MongoDB Atlas
- Mobile App Distribution: Play Store / App Store
- CI/CD: GitHub Actions (optional)

---

4. Module Design

4.1 Authentication Module

Features:

- User Registration
- Login
- Role-based access (Customer / Store Owner / Admin)

Security:

- JWT Token Authentication
- Password Hashing (BCrypt)

4.2 Offer Management Module

Features:

- Add Offer
- Update Offer
- Delete Offer
- Offer Expiry Handling

Database Fields:

- offerId
- title
- description
- discountPercentage
- storeId
- expiryDate
- createdAt

4.3 Cart Module

Features:

- Add to cart

- Remove from cart
- Update quantity
- Calculate total price

Cart stored in:

- Database (persistent)
- OR Local state until checkout

4.4 Payment Module

Integration:

- Razorpay / Stripe API

Flow:

1. User initiates payment
2. Backend generates payment order
3. Payment gateway processes transaction
4. Backend verifies payment
5. Order status updated

Security:

- Server-side payment verification
- Encrypted API keys

4.5 Real-Time Notification Module

Technology:

- Socket.io

Use Cases:

- New offer alert
- Offer expiring soon
- Order status update

---

5. API Design

5.1 Authentication APIs

POST /api/auth/register
POST /api/auth/login

5.2 Offer APIs

GET /api/offers
POST /api/offers
PUT /api/offers/{id}
DELETE /api/offers/{id}

5.3 Cart APIs

POST /api/cart
GET /api/cart
DELETE /api/cart/{id}

5.4 Payment APIs

POST /api/payment/create
POST /api/payment/verify

---

6. Database Design

6.1 Collections

Users

- _id
- name
- email
- password
- role

Stores

- _id
- storeName
- ownerId
- location

Offers

- _id
- title
- description
- discount
- storeId
- expiryDate

Orders

- _id
- userId
- items
- totalAmount
- paymentStatus

---

7. Security Architecture

- HTTPS protocol
- JWT authentication
- Role-based authorization
- Encrypted passwords
- Secure payment gateway
- Input validation & sanitization
- Rate limiting

---

8. Performance Requirements

- API response time < 2 seconds
- Support 10,000+ concurrent users
- Efficient indexing on MongoDB

---

9. Scalability Strategy

- Horizontal scaling using load balancer
- Database sharding (future)
- Microservice split for:
    - Auth
    - Payments
    - Offers

---

10. Error Handling Strategy

- Standardized API response format:
  {
  status,
  message,
  data
  }
- Centralized exception handler
- Proper HTTP status codes

---

11. Logging & Monitoring

- Logging: Log4j / SLF4J
- Monitoring: AWS CloudWatch
- Error Tracking: Sentry (optional)

---

12. Backup & Recovery

- Daily database backup
- Automated snapshot storage
- Disaster recovery plan

---

13. Deployment Strategy

Development → Testing → Production

- Use separate environments
- Environment variables for secrets
- CI/CD pipeline for automatic deployment

---

14. Assumptions & Dependencies

- Online platforms provide APIs
- Store owners actively update offers
- Payment gateway approva